

Лекция №4

Функции в JavaScript

Цель лекции

Научиться создавать и использовать функции, передавать в них данные и получать результат их работы.

- создавать и вызывать функции;
- использовать параметры и аргументы;
- возвращать результат через return;
- понимать область видимости переменных;
- создавать простые стрелочные функции.

1. Что такое функция

Функция — отдельный блок кода для выполнения определённой задачи. Она позволяет не повторять одинаковый код и делить программу на логические части.

```
function showMessage() {  
    console.log("Добро пожаловать!");  
}  
  
showMessage();  
showMessage();
```

2. Создание и вызов функции

Функция создаётся с помощью ключевого слова `function`. Код между фигурными скобками называется телом функции. После объявления функцию необходимо вызвать.

```
function showHello() {  
    console.log("Привет!");  
}  
  
showHello();
```

3. Имена функций

Имя должно отражать выполняемое действие. Обычно используется `camelCase` и глаголы: `showMessage()`, `calculateTotal()`, `checkStock()`, `getDiscount()`.

4. Параметры и аргументы

Параметр — переменная, указанная при объявлении функции. Аргумент — конкретное значение, передаваемое при вызове.

```
function showProduct(name) {  
    console.log(`Товар: ${name}`);  
}  
  
showProduct("Ноутбук");
```

5. Несколько параметров

Функция может принимать несколько параметров. Аргументы передаются в том же порядке, в котором указаны параметры.

```
function showProduct(name, price) {  
    console.log(`Товар: ${name}`);  
    console.log(`Цена: ${price} руб.`);  
}  
  
showProduct("Ноутбук", 65000);
```

6. Вычисления внутри функции

Внутри функции можно выполнять арифметические операции и использовать ранее изученные конструкции.

```
function showTotal(price, count) {  
    const total = price * count;  
    console.log(`Стоимость: ${total} руб.`);  
}
```

```
}  
  
showTotal(1500, 3);
```

7. Возврат результата — return

return возвращает результат работы функции. Полученное значение можно сохранить в переменную и использовать дальше.

```
function calculateTotal(price, count) {  
    return price * count;  
}  
  
const total = calculateTotal(1500, 3);  
console.log(total);
```

8. console.log() и return

console.log() выводит значение в консоль, а return возвращает его из функции. Для вычисляющих функций обычно удобнее использовать return.

```
function calculateTotal(price, count) {  
    return price * count;  
}  
  
const total = calculateTotal(5000, 2);  
  
if (total >= 10000) {  
    console.log("Предоставляется скидка");  
}
```

9. return завершает функцию

После выполнения return функция прекращает работу.

```
function checkStock(stock) {  
    if (stock === 0) {  
        return "Товар отсутствует";  
    }  
  
    return "Товар есть в наличии";  
}
```

10. Функция с условием

В функции можно использовать if, else if и else.

```
function getDiscount(total) {  
    if (total >= 10000) {  
        return 15;  
    } else if (total >= 5000) {  
        return 10;  
    } else {  
        return 0;  
    }  
}
```

11. Функция с циклом

Внутри функции можно использовать цикл.

```
function showNumbers(count) {  
    for (let i = 1; i <= count; i++) {  
        console.log(i);  
    }  
}  
  
showNumbers(5);
```

12. Область видимости

Переменная, созданная внутри функции, обычно доступна только внутри неё. Такая переменная называется локальной.

```
function calculateTotal(price, count) {  
    const total = price * count;
```

```
    console.log(total);  
}
```

13. Внешние переменные

Функция может обращаться к внешним переменным, но необходимые данные обычно лучше передавать через параметры. Так функция меньше зависит от остальной программы.

```
function showTax(tax) {  
    console.log(tax);  
}
```

```
showTax(20);
```

14. Значение параметра по умолчанию

Параметру можно задать значение, которое будет использовано, если аргумент не передан.

```
function showUser(name = "Гость") {  
    console.log(`Пользователь: ${name}`);  
}
```

```
showUser("Анна");  
showUser();
```

15. Функциональное выражение

Функцию можно сохранить в переменную. Такая запись называется функциональным выражением.

```
const calculateTotal = function(price, count) {  
    return price * count;  
};
```

```
console.log(calculateTotal(2000, 3));
```

16. Стрелочные функции

Стрелочная функция — современная компактная форма записи функции.

```
const calculateTotal = (price, count) => {  
    return price * count;  
};
```

17. Короткая запись стрелочной функции

Если функция только возвращает одно выражение, фигурные скобки и return можно опустить.

```
const calculateTotal = (price, count) => price * count;  
const double = (number) => number * 2;
```

18. Обычная или стрелочная функция

Оба варианта используются в JavaScript. На начальном этапе основные функции удобно создавать через function. Стрелочные функции особенно часто встретятся при изучении массивов, Node.js и асинхронного программирования.

19. Комплексный пример «Заказ»

Разделим обработку заказа на несколько небольших функций.

```
function calculateTotal(price, count) {  
    return price * count;  
}  
  
function getDiscount(total) {  
    if (total >= 10000) return 10;  
    return 0;  
}  
  
function calculateFinalPrice(total, discount) {  
    return total - total * discount / 100;  
}  
  
const total = calculateTotal(3000, 4);  
const discount = getDiscount(total);  
const finalPrice = calculateFinalPrice(total, discount);  
  
console.log(`К оплате: ${finalPrice} руб.`);
```

20. Типичные ошибки

Частые ошибки: функция объявлена, но не вызвана; `console.log()` используется вместо `return`; нарушен порядок аргументов; выполняется обращение к локальной переменной вне функции.

Краткие выводы

Функции позволяют выделять отдельные задачи программы в самостоятельные блоки. Основные понятия: `function`, параметры, аргументы, `return` и область видимости. JavaScript также поддерживает функциональные выражения и стрелочные функции. Функции являются одной из основных конструкций JavaScript и Node.js.